

ASPA – приложение ASP.NET CORE

♣ – задания, требующие самостоятельного изучения (нет в лекциях)

♣ **Задание 1.**

1. Установите программную платформу **POSTMAN**

<https://www.postman.com/>

<https://habr.com/ru/companies/maxilect/articles/596789/>

2. Убедитесь в работоспособности платформы.

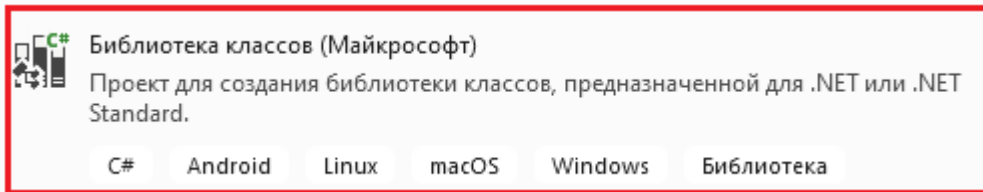
Задание 2.

3. Разработайте библиотеку, применив следующий шаблон

Имя решения: **ASPA**

Имя проекта: **DAL004**

Версия .NET: **7.0** или **8.0**



4. Библиотека **DAL004** должна реализовать **DAL** (Data Access Level) реализовав паттерн **Repository** для доступа к следующим формату JSON (формат, данные и принцип их размещения описаны в лабораторной работе 3).

5. **Repository** должен реализовывать следующий интерфейс (расширен относительно **DAL003**).

```
public interface IRepository:IDisposable
{
    string      BasePath { get; } // полный директорий для JSON и фотографий
    Celebrity[] GetAllCelebrities(); // получить весь список знаменитостей
    Celebrity?  GetCelebrityById(int id); // получить знаменитость по Id
    Celebrity[] GetCelebritiesBySurname(string Surname); // получить знаменитость по фамилии
    string?     GetPhotoPathById(int id); // получить путь для GET-запроса к фотографии
    int?        AddCelebrity(Celebrity celebrity); // добавить знаменитость, =Id новой знаменитости
    bool        DelCelebrityById(int id); // удалить знаменитость по Id, =true - успех
    int?        UpdCelebrityById(int id, Celebrity celebrity); // изменить знаменитость по Id, =Id - новый Id - успех
    int         SaveChanges(); // сохранить изменения в JSON, =количество изменений
}
```

6. Разработайте консольное приложение, применив следующий шаблон

Имя решения: **ASPA**

Имя проекта: **Test_DAL004**

Версия .NET: **7.0** или **8.0**



Консольное приложение (Майкрософт)

Проект для создания приложения командной строки, которое может выполняться в среде .NET в Windows, Linux и macOS

C#

Linux

macOS

Windows

Консоль

7. Приложение **Test_DAL004** предназначено для тестирования новых функций **DAL004**. Ниже приведён код и результат теста

```
Repository.JSONFileName = "Celebrities.json";           // имя файла
using (IRepository repository = Repository.Create("Celebrities")) // директорий (в текущем директории приложения)
{
    void print(string label)
    {
        Console.WriteLine("--- "+label+ " -----");
        foreach (Celebrity celebrity in repository.getAllCelebrities())
        {
            Console.WriteLine($"Id = {celebrity.Id}, Firstname = {celebrity.Firstname}, " +
                               $"Surname = {celebrity.Surname}, PhotoPath = {celebrity.PhotoPath} ");
        }
    }

    print("start");

    int? testdel1 = repository.addCelebrity(new Celebrity(0, "TestDel1", "TestDel1", "Photo/TestDel1.jpg"));
    int? testdel2 = repository.addCelebrity(new Celebrity(0, "TestDel2", "TestDel2", "Photo/TestDel2.jpg"));
    int? testupd1 = repository.addCelebrity(new Celebrity(0, "TestUpd1", "TestUpd1", "Photo/TestUpd1.jpg"));
    int? testupd2 = repository.addCelebrity(new Celebrity(0, "TestUpd2", "TestUpd2", "Photo/TestUpd2.jpg"));
    repository.SaveChanges();
    print("add 4");

    if (testdel1 != null)
        if (repository.delCelebrityById((int)testdel1)) Console.WriteLine($" delete {testdel1} "); else Console.WriteLine($"delete {testdel1} error");
    if (testdel2 != null)
        if (repository.delCelebrityById((int)testdel2)) Console.WriteLine($" delete {testdel2} "); else Console.WriteLine($"delete {testdel2} error");
    if (repository.delCelebrityById(1000)) Console.WriteLine($" delete {1000} "); else Console.WriteLine($"delete {1000} error");
    repository.SaveChanges();
    print("del 2");

    if (testupd1 != null)
        if (repository.updCelebrityById((int)testupd1, new Celebrity(0, "Updated1", "Updated1", "Photo/Updated1.jpg"))) Console.WriteLine($" update {testupd1} ");
        else Console.WriteLine($"update {testupd1} error");
    if (testupd2 != null)
        if (repository.updCelebrityById((int)testupd2, new Celebrity(0, "Updated2", "Updated2", "Photo/Updated2.jpg"))) Console.WriteLine($" update {testupd2} ");
        else Console.WriteLine($"update {testupd2} error");
    if (repository.updCelebrityById(1000, new Celebrity(0, "Updated1000", "Updated1000", "Photo/Updated1000.jpg"))) Console.WriteLine($" update {1000} ");
    else Console.WriteLine($"update {1000} error");
    repository.SaveChanges();
    print("upd 2");
}
```

```

D:\VS_Projects\ANC\Test_DAL004\bin\Debug\net7.0\Test_DAL004.exe
--- start -----
Id = 1, Firstname = Noam, Surname = Chomsky, PhotoPath = /Photo/Chomsky.jpg
Id = 2, Firstname = Tim, Surname = Berners-Lee, PhotoPath = /Photo/Berners-Lee.jpg
Id = 3, Firstname = Edgar, Surname = Codd, PhotoPath = /Photo/Codd.jpg
Id = 4, Firstname = Donald, Surname = Knuth, PhotoPath = /Photo/Knuth.jpg
Id = 5, Firstname = Linus, Surname = Torvalds, PhotoPath = /Photo/Torvalds.jpg
Id = 6, Firstname = John, Surname = Neumann, PhotoPath = /Photo/Neumann.jpg
Id = 7, Firstname = Edsger, Surname = Dijkstra, PhotoPath = /Photo/Dijkstra.jpg
--- add 4 -----
Id = 1, Firstname = Noam, Surname = Chomsky, PhotoPath = /Photo/Chomsky.jpg
Id = 2, Firstname = Tim, Surname = Berners-Lee, PhotoPath = /Photo/Berners-Lee.jpg
Id = 3, Firstname = Edgar, Surname = Codd, PhotoPath = /Photo/Codd.jpg
Id = 4, Firstname = Donald, Surname = Knuth, PhotoPath = /Photo/Knuth.jpg
Id = 5, Firstname = Linus, Surname = Torvalds, PhotoPath = /Photo/Torvalds.jpg
Id = 6, Firstname = John, Surname = Neumann, PhotoPath = /Photo/Neumann.jpg
Id = 7, Firstname = Edsger, Surname = Dijkstra, PhotoPath = /Photo/Dijkstra.jpg
Id = 8, Firstname = TestDel1, Surname = TestDel1, PhotoPath = Photo/TestDel1.jpg
Id = 9, Firstname = TestDel2, Surname = TestDel2, PhotoPath = Photo/TestDel2.jpg
Id = 10, Firstname = TestUpd1, Surname = TestUpd1, PhotoPath = Photo/TestUpd1.jpg
Id = 11, Firstname = TestUpd2, Surname = TestUpd2, PhotoPath = Photo/TestUpd2.jpg
delete 8
delete 9
delete 1000 error
--- del 2 -----
Id = 1, Firstname = Noam, Surname = Chomsky, PhotoPath = /Photo/Chomsky.jpg
Id = 2, Firstname = Tim, Surname = Berners-Lee, PhotoPath = /Photo/Berners-Lee.jpg
Id = 3, Firstname = Edgar, Surname = Codd, PhotoPath = /Photo/Codd.jpg
Id = 4, Firstname = Donald, Surname = Knuth, PhotoPath = /Photo/Knuth.jpg
Id = 5, Firstname = Linus, Surname = Torvalds, PhotoPath = /Photo/Torvalds.jpg
Id = 6, Firstname = John, Surname = Neumann, PhotoPath = /Photo/Neumann.jpg
Id = 7, Firstname = Edsger, Surname = Dijkstra, PhotoPath = /Photo/Dijkstra.jpg
Id = 10, Firstname = TestUpd1, Surname = TestUpd1, PhotoPath = Photo/TestUpd1.jpg
Id = 11, Firstname = TestUpd2, Surname = TestUpd2, PhotoPath = Photo/TestUpd2.jpg
update 10
update 11
update 1000 error
--- upd 2 -----
Id = 1, Firstname = Noam, Surname = Chomsky, PhotoPath = /Photo/Chomsky.jpg
Id = 2, Firstname = Tim, Surname = Berners-Lee, PhotoPath = /Photo/Berners-Lee.jpg
Id = 3, Firstname = Edgar, Surname = Codd, PhotoPath = /Photo/Codd.jpg
Id = 4, Firstname = Donald, Surname = Knuth, PhotoPath = /Photo/Knuth.jpg
Id = 5, Firstname = Linus, Surname = Torvalds, PhotoPath = /Photo/Torvalds.jpg
Id = 6, Firstname = John, Surname = Neumann, PhotoPath = /Photo/Neumann.jpg
Id = 7, Firstname = Edsger, Surname = Dijkstra, PhotoPath = /Photo/Dijkstra.jpg
Id = 12, Firstname = Updated1, Surname = Updated1, PhotoPath = Photo/Updated1.jpg
Id = 13, Firstname = Updated2, Surname = Updated2, PhotoPath = Photo/Updated2.jpg

```

8. Выполните тест **Test_DAL004** и убедитесь в его успешном выполнении.

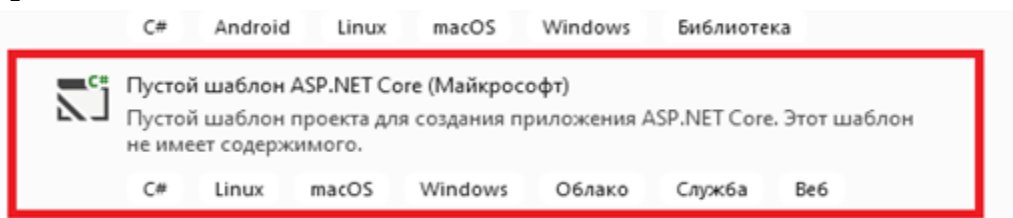
Задание 3.

9. Разработайте ASPA, применив следующий шаблон.

Имя решения: **ASPA**

Имя проекта: **ASPA004_1**

Версия .NET: **7.0** или **8.0**



10. Подключите библиотеку **DAL004** к **ASPA004_1**

11. Исследуйте приведённый ниже фрагмент приложения и включите в **ASPA004_1**. Поясните назначение подчёркнутых или обведённых фрагментов программного кода.

```

Repository.JSONFileName = "Celebrities.json";
using (IRepository repository = Repository.Create("Celebrities"))
{
    app.UseExceptionHandler("/Celebrities/Error");

    app.MapGet("/Celebrities", () => repository.getAllCelebrities()); // ASPA03
    app.MapGet("/Celebrities/{id:int}", (int id) => {
        Celebrity? celebrity = repository.getCelebrityById(id);
        if (celebrity == null) throw new FoundByIdException($"Celebrity Id = {id}");
        return celebrity;
    });
    app.MapPost("/Celebrities", (Celebrity celebrity) => {
        int? id = repository.addCelebrity(celebrity);
        if (id == null) throw new AddCelebrityException("/Celebrities error, id == null");
        if (repository.SaveChanges() <= 0) throw new SaveException("/Celebrities error, SaveChanges() <= 0");
        return new Celebrity((int)id, celebrity.Firstname, celebrity.Surname, celebrity.PhotoPath);
    });

    app.MapFallback((HttpContext ctx) => Results.NotFound(new { error = $"path {ctx.Request.Path} not supported" }));

    app.Map("/Celebrities/Error", (HttpContext ctx) => {
        Exception? ex = ctx.Features.Get<ExceptionHandlerFeature>()?.Error;
        IResult rc = Results.Problem(detail: "Panic", instance: app.Environment.EnvironmentName, title: "ASPA004", statusCode: 500);
        if (ex != null)
        {
            if (ex is FoundByIdException) rc = Results.NotFound(ex.Message); // не найден 404
            if (ex is BadRequestException) rc = Results.BadRequest(ex.Message); // ошибка в формате запроса 400
            if (ex is SaveException) rc = Results.Problem(title: "ASPA004/SaveChanges", detail: ex.Message, instance: app.Environment.EnvironmentName, statusCode: 500);
            if (ex is AddCelebrityException) rc = Results.Problem(title: "ASPA004/addCelebrity", detail: ex.Message, instance: app.Environment.EnvironmentName, statusCode: 500);
        }
        return rc;
    });
}

app.Run();
}

public class FoundByIdException:Exception {public FoundByIdException(string message) : base($"Found by Id: {message}") { } };
public class SaveException:Exception {public SaveException(string message):base($"SaveChanges error:{message}") { } };
public class AddCelebrityException: Exception {public AddCelebrityException(string message):base($"AddCelebrityException error:{message}") { } };

```

12. Выполните следующие запросы с помощью **POSTMAN** и сравните полученные ответы. Поясните: какие фрагменты кода принимали участие в обработке каждого запроса и формировании ответа? Определите статус каждого ответа в Postman, а также где он задаётся в программном коде и формируется соответствующее сообщение.

GET http://localhost:5050/Celebrities

Params Authorization Headers (12) Body ● Scripts Settings

Body Cookies Headers (4) Test Results ↺

{ } JSON ▾ ▶ Preview 🔄 Visualize ▾

```
1  [
2    {
3      "id": 1,
4      "firstname": "Noam",
5      "surname": "Chomsky",
6      "photoPath": "/Photo/Chomsky"
7    },
8    {
9      "id": 2,
10     "firstname": "Tim",
11     "surname": "Berners-Lee",
12     "photoPath": "/Photo/Berners-Lee"
13   },
14   ...
15 ]
```

```
32   {
33     "id": 6,
34     "firstname": "John",
35     "surname": "Neumann",
36     "photoPath": "/Photo/Neumann"
37   },
38   {
39     "id": 7,
40     "firstname": "Edsger",
41     "surname": "Dijkstra",
42     "photoPath": "/Photo/Dijkstra"
43   }
44 ]
```

13.

GET http://localhost:5050/Celebrities/5

Params Authorization Headers (12) Body ● Scripts Settings

Body Cookies Headers (4) Test Results ↺

{ } JSON ▾ ▶ Preview 🔄 Visualize ▾

```
1  {
2    "id": 5,
3    "firstname": "Linus",
4    "surname": "Torvalds",
5    "photoPath": "/Photo/Torvalds"
6  }
```

14.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** <http://localhost:5050/Celebrities/5555>
- Body:** `1 "Found by Id: Celebrity Id = 5555"`

15.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** <http://localhost:5050/Celebrities/5aa4>
- Body:**

```
1 {
2   "error": "path /Celebrities/5aa4 not supported"
3 }
```

16.

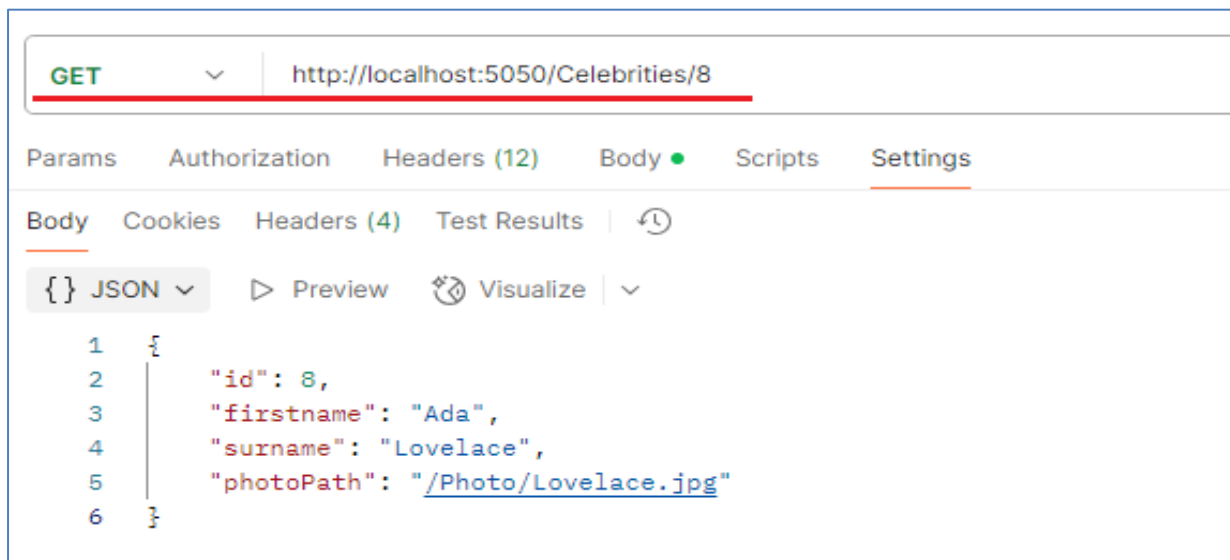
The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <http://localhost:5050/Celebrities>
- Body Type:** raw
- Body:**

```
1 {
2   "firstname": "Ada",
3   "surname": "Lovelace",
4   "photoPath": "/Photo/Lovelace.jpg"
5 }
```
- Response Body:**

```
1 {
2   "id": 8,
3   "firstname": "Ada",
4   "surname": "Lovelace",
5   "photoPath": "/Photo/Lovelace.jpg"
6 }
```

17.



GET <http://localhost:5050/Celebrities/8>

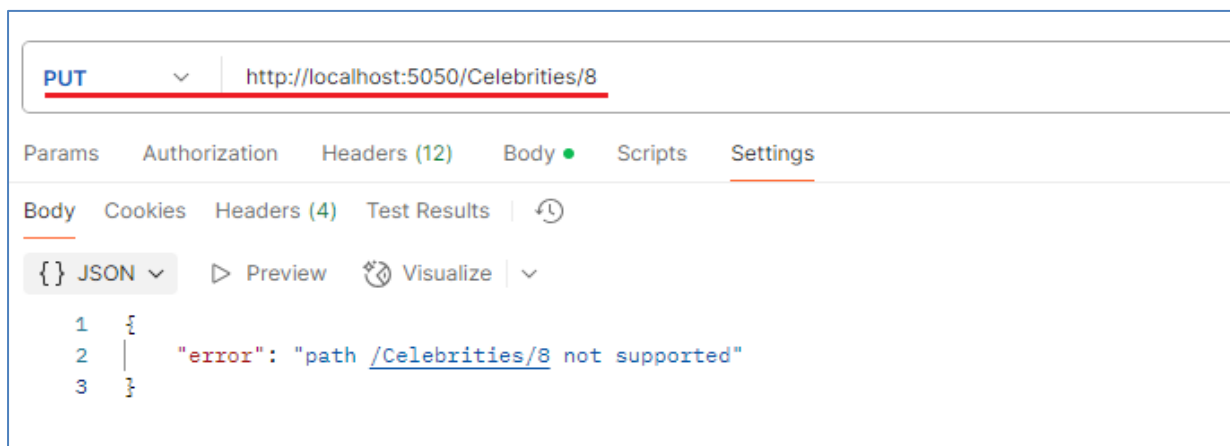
Params Authorization Headers (12) Body ● Scripts Settings

Body Cookies Headers (4) Test Results | ↺

{ } JSON ▾ ▶ Preview 📄 Visualize ▾

```
1 {
2   "id": 8,
3   "firstname": "Ada",
4   "surname": "Lovelace",
5   "photoPath": "/Photo/Lovelace.jpg"
6 }
```

18.



PUT <http://localhost:5050/Celebrities/8>

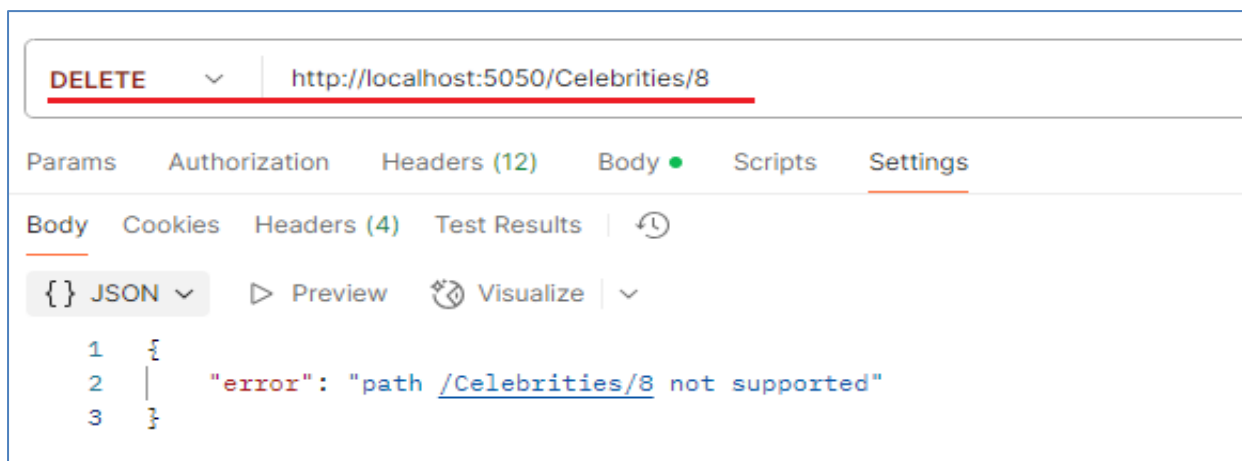
Params Authorization Headers (12) Body ● Scripts Settings

Body Cookies Headers (4) Test Results | ↺

{ } JSON ▾ ▶ Preview 📄 Visualize ▾

```
1 {
2   "error": "path /Celebrities/8 not supported"
3 }
```

19.



DELETE <http://localhost:5050/Celebrities/8>

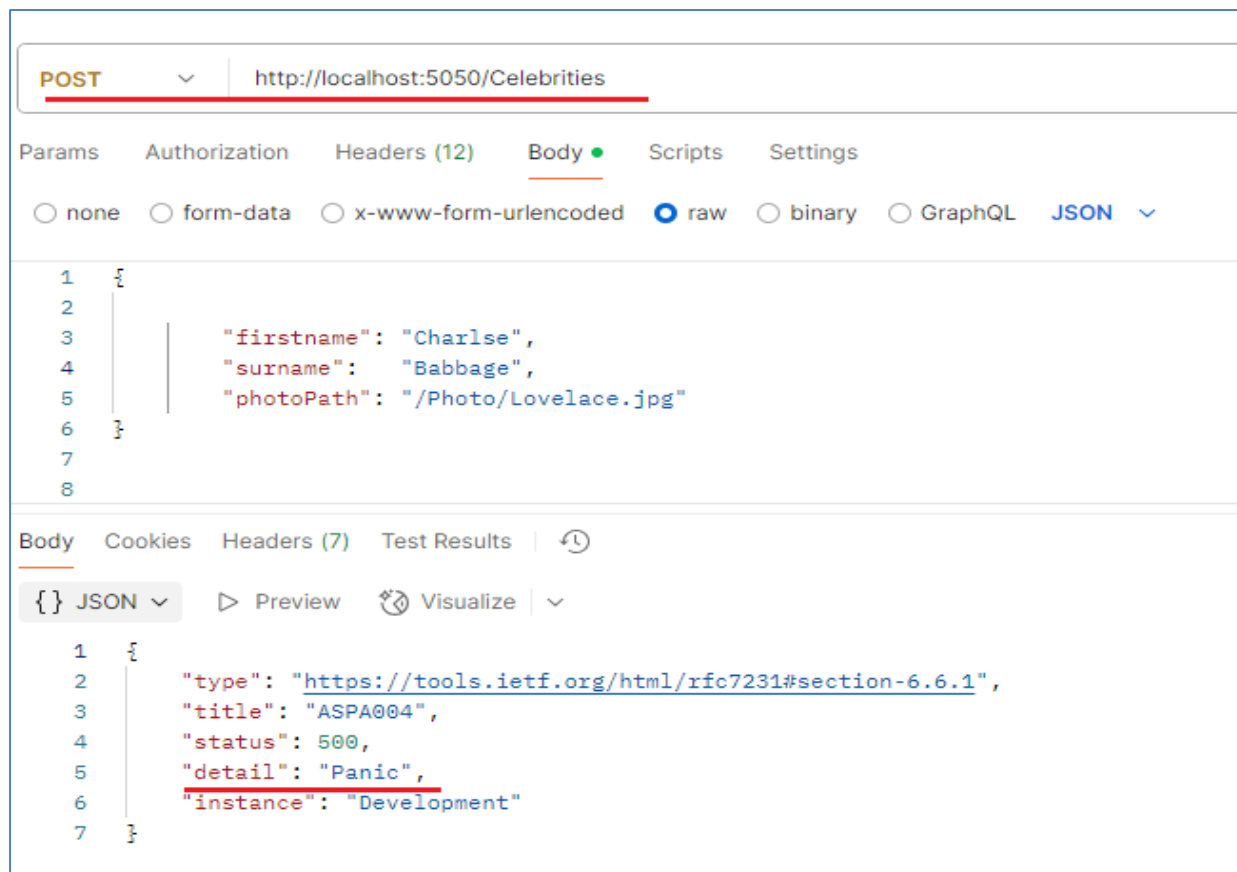
Params Authorization Headers (12) Body ● Scripts Settings

Body Cookies Headers (4) Test Results | ↺

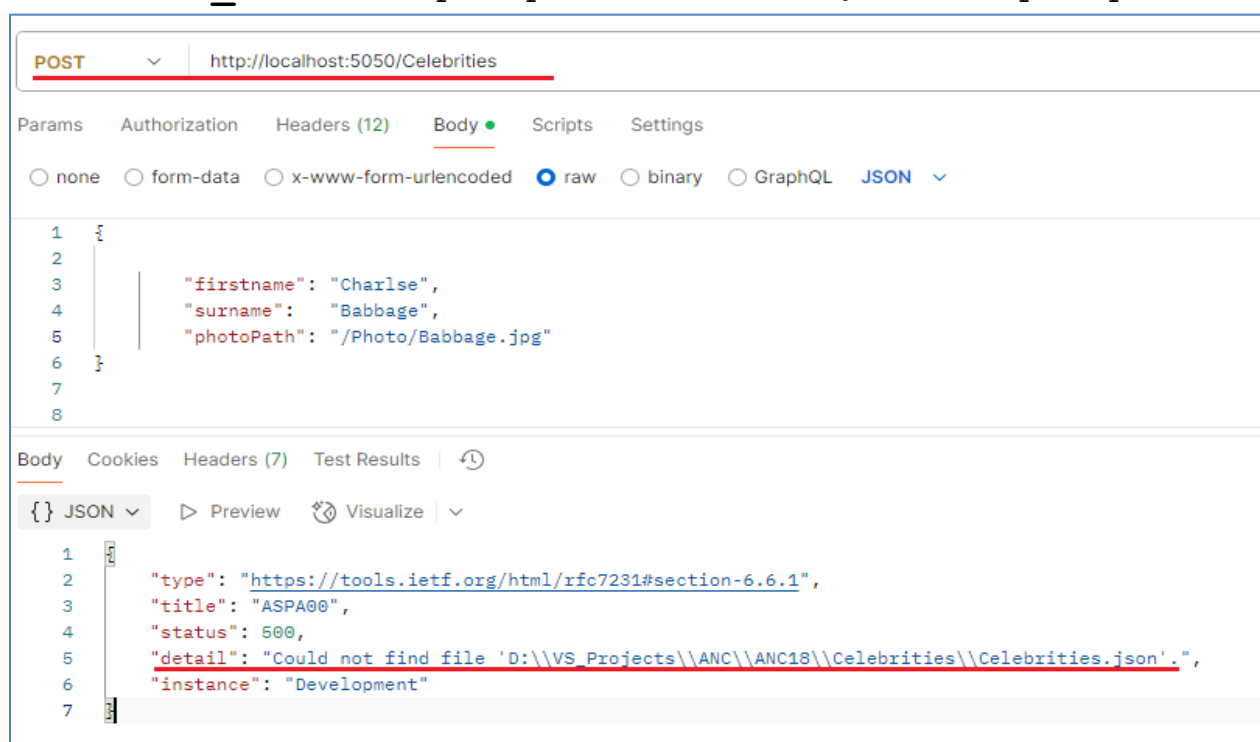
{ } JSON ▾ ▶ Preview 📄 Visualize ▾

```
1 {
2   "error": "path /Celebrities/8 not supported"
3 }
```

20. Не останавливая **ASPA004_1**, выполните действия, приводящие к следующему поведению **ASPA004_1**.



21. Выполните анализ протокола, выводимого на консоль в предыдущем пункте задания. Предложите такие изменение в программном коде, чтобы в поле **detail**, возвращаемого в ответе сообщения, была более подробная информация об ошибке. Измените программный код **ASPA004_1** так, чтобы поведение **ASPA004_1** стало примерно как в следующем примере.



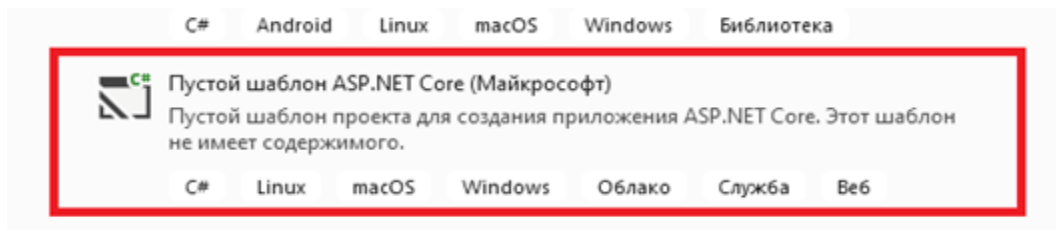
Задание 3.

22. Разработайте ASPA, применив следующий шаблон.

Имя решения: **ASPA**

Имя проекта: **ASPA004_2**

Версия .NET: **7.0** или **8.0**



23. Подключите библиотеку **DAL004** к **ASPA004 2**.

24. Продублируйте программный код файла **Program.cs** приложения **ASPA004 1** в одноименный файл приложения **ASPA004 2**.

25. Добавьте в **Program.cs** фрагмент кода для обработки DELETE-запросов. Зачёркнутый код должен быть написан вами.

```
app.MapDelete("/Celebrities/{id:int}", (int id) => {
    // if (id < 1 || id > 1000000000) return new BadRequestResult();
    // return BadRequest();
});
```

26. Следующие два фрагмента программного кода уже существуют в **Program.cs**. Необходимо вставить собственный программный код вместо зачёркнутых строк

```
app.Map("/Celebrities/Error", (HttpContext ctx) => {
    Exception? ex = ctx.Features.Get<ExceptionHandlerFeature>()?.Error;
    IResult rc = Results.Problem(detail: "Panic", instance: app.Environment.EnvironmentName, title: "ASPA004", statusCode: 500);
    if (ex != null)
    {
        if (ex is FileNotFoundException) rc = Results.Problem(title: "ASPA000", detail: ex.Message, instance: app.Environment.EnvironmentName, statusCode: 500);
        if (ex is FileNotFoundException) rc = Results.Problem(title: "ASPA000", detail: ex.Message, instance: app.Environment.EnvironmentName, statusCode: 500);
        if (ex is FoundByIdException) rc = Results.NotFound(ex.Message); // не найден 404
        if (ex is BadRequestException) rc = Results.BadRequest(ex.Message); // ошибка в формате запроса 400
        if (ex is SaveException) rc = Results.Problem(title: "ASPA004/SaveChanges", detail: ex.Message, instance: app.Environment.EnvironmentName, statusCode: 500);
        if (ex is AddCelebrityException) rc = Results.Problem(title: "ASPA004/addCelebrity", detail: ex.Message, instance: app.Environment.EnvironmentName, statusCode: 500);
    }
    return rc;
});
```

```
public class FoundByIdException:Exception {public FoundByIdException(string message) :base($"Found by Id: {message}") { } }
public class SaveException:Exception {public SaveException(string message) :base($"SaveChanges error:{message}") { } }
public class AddCelebrityException: Exception {public AddCelebrityException(string message) :base($"AddCelebrityException error:{message}") { } };
```

27. Окончательный вариант **ASPA004_2** должен обеспечивать успешное прохождение следующих 5 тестов.

POST ▼ | http://localhost:5050/Celebrities

Params Authorization Headers (12) **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   |
3   |     "firstname": "ForDelete",
4   |     "surname": "ForDelete",
5   |     "photoPath": "/Photo/ForDelete.jpg"
6   | }
7
8
```

Body Cookies Headers (4) Test Results ↺

{ } JSON ▼ ▶ Preview 🔗 Visualize ▼

```
1 {
2   |
3   |     "id": 9,
4   |     "firstname": "ForDelete",
5   |     "surname": "ForDelete",
6   |     "photoPath": "/Photo/ForDelete.jpg"
7   | }
8
```

GET ▼ | http://localhost:5050/Celebrities/9

Params Authorization Headers (12) **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

Body Cookies Headers (4) Test Results ↺

{ } JSON ▼ ▶ Preview 🔗 Visualize ▼

200 OK ●

```
1 {
2   |
3   |     "id": 9,
4   |     "firstname": "ForDelete",
5   |     "surname": "ForDelete",
6   |     "photoPath": "/Photo/ForDelete.jpg"
7   | }
8
```

DELETE

http://localhost:5050/Celebrities/9

ParamsAuthorizationHeaders (7)BodyScriptsSettings

Query Params

	Key
	Key

BodyCookiesHeaders (4)Test Results200 OK

{ } JSON

Preview

Visualize

1 "Celebrity with Id = 9 deleted"

GET

http://localhost:5050/Celebrities/9

ParamsAuthorizationHeaders (7)BodyScriptsSettings

Query Params

	Key	Value
	Key	Value

BodyCookiesHeaders (7)Test Results404 Not Found

{ } JSON

Preview

Visualize

1 "Found by Id: /Celebrities, Celebrity Id = 9"

DELETE

http://localhost:5050/Celebrities/1000

ParamsAuthorizationHeaders (7)BodyScriptsSettings

Query Params

	Key	Value
	Key	Value

BodyCookiesHeaders (7)Test Results404 Not Found

{ } JSON

Preview

Visualize

1 "Delete by Id:DELETE /Celebrities error, Id = 1000"

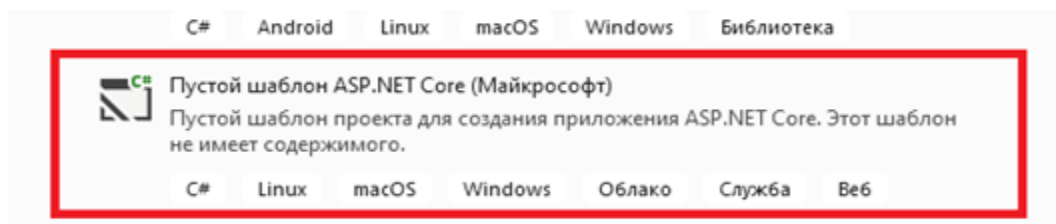
Задание 4

28. Разработайте ASPA, применив следующий шаблон.

Имя решения: **ASPA**

Имя проекта: **ASPA004_3**

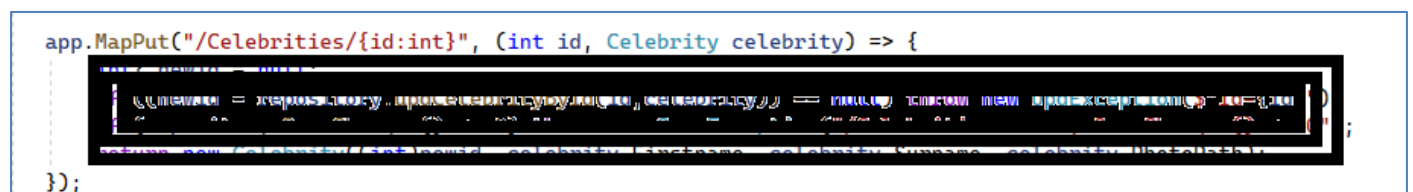
Версия .NET: **7.0** или **8.0**



29. Подключите библиотеку **DAL004** к **ASPA004_3**.

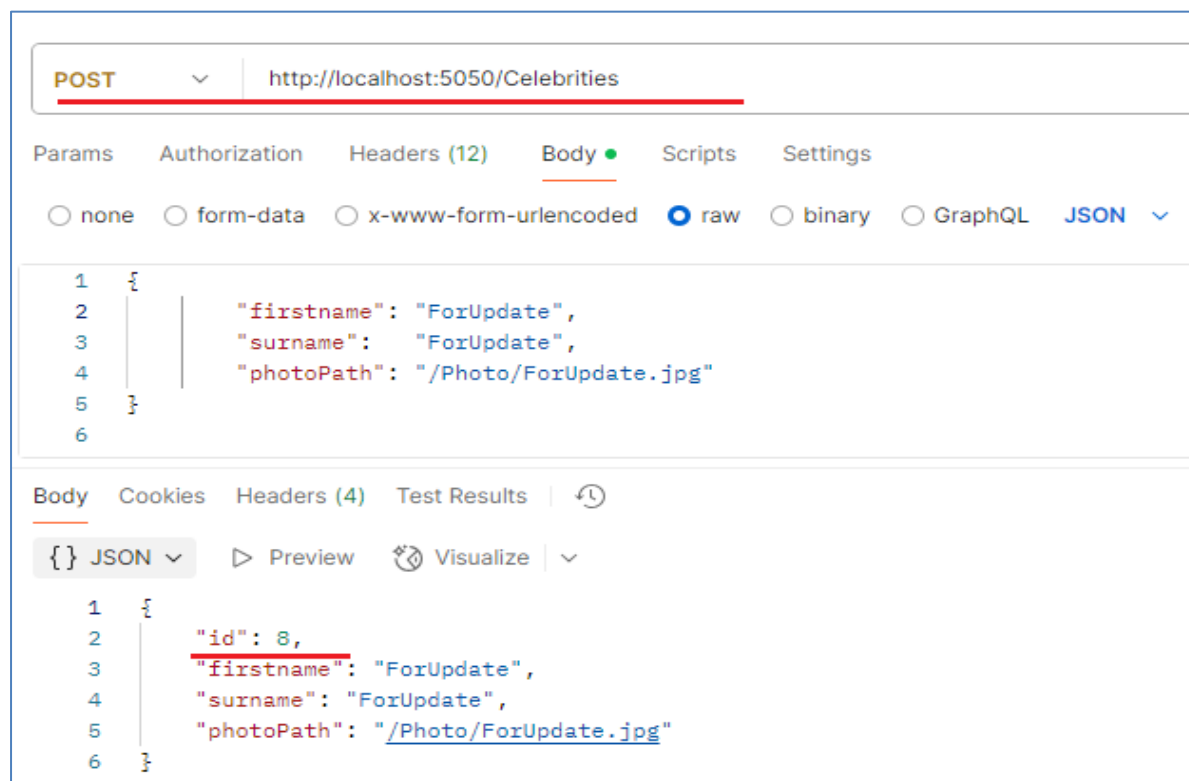
30. Продублируйте программный код файла **Program.cs** приложения **ASPA004_2** в одноименный файл приложения **ASPA004_3**.

31. Добавьте в **Program.cs** фрагмент кода для обработки PUT-запросов. Зачёркнутый код должен быть написан вами.



32. Другие два фрагмента, в которые необходимо дописать код совпадают с фрагментами, используемыми при разработке **ASPA004_2**.

33. Окончательный вариант **ASPA004_3** должен обеспечивать успешное прохождение следующих 3 тестов.



PUT ▼ http://localhost:5050/Celebrities/8

Params Authorization Headers (9) **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "firstname": "Updated",
3   "surname": "Updated",
4   "photoPath": "/Photo/Updated.jpg"
5 }
```

Body Cookies Headers (4) Test Results ↺

{} JSON ▼ ▶ Preview 🔗 Visualize ▼

```
1 {
2   "id": 8,
3   "firstname": "Updated",
4   "surname": "Updated",
5   "photoPath": "/Photo/Updated.jpg"
6 }
```

GET ▼ http://localhost:5050/Celebrities/8

Params Authorization Headers (12) **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

1

Body Cookies Headers (4) Test Results ↺

{} JSON ▼ ▶ Preview 🔗 Visualize ▼

```
1 {
2   "id": 8,
3   "firstname": "Updated",
4   "surname": "Updated",
5   "photoPath": "/Photo/Updated.jpg"
6 }
```

Задание 5. Ответьте на следующие вопросы

34. Поясните назначение следующих функций

```
// MapGet()
// MapPost()
// MapPut()
// MapDelete()
// MapFallback()
// UseExceptionHandler()
// UseStaticFiles()
//
```

35. Поясните назначение следующих объектов и интерфейсов

```
// IResult
// Exception
// IDisposable
// Results.Problem и RFC7231, section 6.6.1
// HttpContext
```

36. ♣ Чем известны Ada Lovelace и Charles Babbage? 😊